Sri Lanka Institute of Information Technology

IT4010
Research Project

25-26J-451

Checklist 03

Registration Number	Name
IT22056320	Senavirathna K.M.U.T
IT22060662	Jameela M.J.F
IT22562524	ARIYASENA M.K.M.M
IT22076052	Mahindarathna D.G.P.T.D

Deployment Report

1. Introduction

This document describes the deployment architecture and infrastructure used for the SpringForge backend system. The deployment process is designed to ensure scalability, maintainability, and automation using containerization and cloud infrastructure.

The backend services are deployed using Docker containers on a cloud virtual machine. Continuous integration and deployment are implemented using GitHub Actions to automate the build and deployment pipeline. A reverse proxy server is used to route requests to the appropriate backend services.

2. System Architecture

The deployed system follows a microservice based architecture where multiple backend services operate independently within containerized environments.

The architecture consists of the following components:

- Frontend Application
  Hosted on a Vercel and accessed via HTTPS.

- Cloud Virtual Machine
  Hosts the backend containers and reverse proxy.

- Container Registry
  Stores Docker images used to deploy backend services.

- Docker Engine
  Runs containerized backend services.

- Nginx Reverse Proxy
  Routes incoming HTTP requests to the appropriate container.

- Continuous Integration and Deployment
  Automates container build and deployment.

3. Cloud Infrastructure

The backend services are deployed on a cloud virtual machine.

- Cloud Provider
  Microsoft Azure

Virtual Machine Configuration

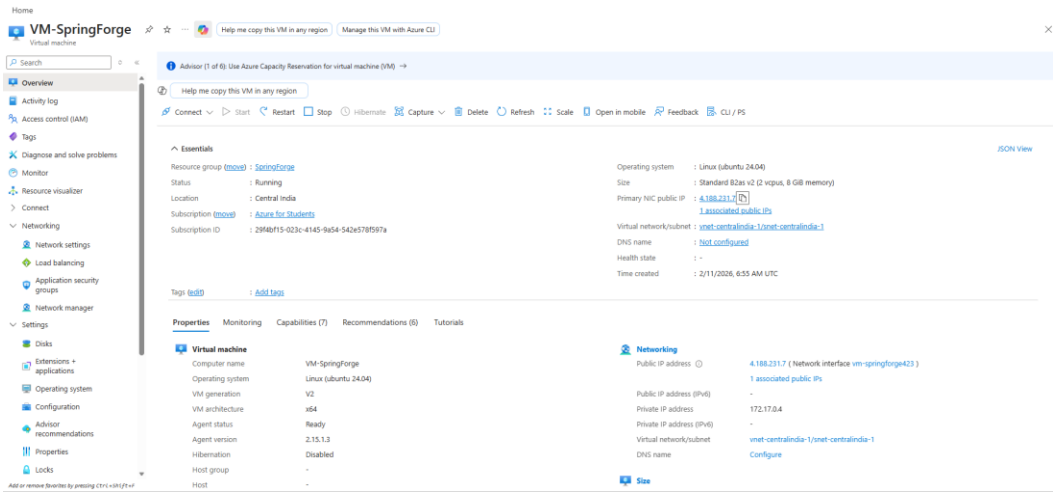
- Operating System
  Ubuntu Server 24.04 LTS
- Instance Type
  Standard B2as v2

Resources

- CPU
  2 Virtual CPUs
- Memory
  4 GB RAM
- Storage
  Standard SSD

Network Configuration

- Port 22
  SSH access
- Port 80
  HTTP access
- Port 443
  HTTPS access



4. Software Environment

The following software components are installed on the virtual machine.

- Docker
  Container runtime for running backend services.
- Docker Compose
  Container orchestration tool used to manage multiple containers.
- Nginx
  Reverse proxy server used to route API requests.
- Python
  Backend runtime environment for machine learning services.
- Certbot
  Used to generate SSL certificates for HTTPS communication.

```
dushxn@VM-SpringForge:~/springforge-development$ docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
dushxn@VM-SpringForge:~/springforge-development$ docker compose version
Docker Compose version v5.1.0
dushxn@VM-SpringForge:~/springforge-development$ nginx -v
nginx version: nginx/1.24.0 (Ubuntu)
dushxn@VM-SpringForge:~/springforge-development$ python3 --version
Python 3.12.3
dushxn@VM-SpringForge:~/springforge-development$ certbot --version
certbot 2.9.0
dushxn@VM-SpringForge:~/springforge-development$ systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
     Active: active (running) since Sun 2026-03-08 05:36:14 UTC; 11h ago
       Docs: man:nginx(8)
    Process: 736 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
    Process: 790 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Main PID: 795 (nginx)
      Tasks: 3 (limit: 9502)
     Memory: 5.1M (peak: 32.1M)
        CPU: 639ms
     CGroup: /system.slice/nginx.service
             ├─795 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             ├─796 "nginx: worker process"
             └─797 "nginx: worker process"

Mar 08 05:36:13 VM-SpringForge systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy s>
Mar 08 05:36:14 VM-SpringForge systemd[1]: Started nginx.service - A high performance web server and a reverse proxy se>
lines 1-17/17 (END)...skipping...
● nginx.service - A high performance web server and a reverse proxy server
```

5. Backend Service Deployment

The backend system is composed of multiple microservices responsible for different functionalities.

Architecture Classification Service

Responsible for classifying software architecture patterns using machine learning models.

- Container Port
  8000

Runtime Error Analysis Service

Analyzes runtime errors and provides suggestions using language models and retrieval augmented generation.

- Container Port
  8082

Code Quality Analysis Service

Evaluates code quality metrics using machine learning models.

- Container Port
  8081

Website backend Service

Backend for the public website which allows users to register and download the p.

- Container Port
  8081

```yaml
version: "3.9"

services:

  architecture-backend:
    image: springforgeacr.azurecr.io/codegeneration-backend:latest
    container_name: architecture-backend

    ports:
      - "8000:8000"

    volumes:
      - /home/dushxn/ml-backend/architecture_model:/app/architecture_model

    restart: always

  rag-error-engine:
    image: springforgeacr.azurecr.io/rag-error-engine:latest
    container_name: rag-error-engine

    ports:
      - "8082:8000"
    env_file:
      - ./rag.env
    restart: always

  code-quality-ml:
    image: springforgeacr.azurecr.io/code-quality:latest
    container_name: code-quality

    ports:
      - "8081:8081"
    env_file:
      - ./quality.env
    restart: always

  springforge-website:
    image: springforgeacr.azurecr.io/website:latest
    container_name: website-backend

    ports:
      - "8085:4000"
    env_file:
      - ./website.env
    restart: always
```

6. Containerization Strategy

Docker is used to containerize the backend services.

Each service includes

- Application source code
- Python dependencies
- Model caching layers
- Runtime configuration

Docker images are built automatically through the CI/CD pipeline and stored in a container registry.

Large machine learning model files are not stored in the repository due to size limitations. Instead, they are stored locally on the virtual machine and mounted into containers using Docker volumes.

Example model path

Host Path
/home/user/ml-backend/architecture_model

Container Path
/app/architecture_model

This approach reduces container size and improves deployment performance.

7. Reverse Proxy Configuration

Nginx is used as a reverse proxy to route incoming requests to the appropriate backend container.

Example routing structure

/code → Architecture classification service
/runtime → Runtime error analysis service
/quality → Code quality analysis service

Incoming requests are received by Nginx and forwarded internally to the corresponding container ports.

Example routing logic

/code requests are forwarded to port 8000
/runtime requests are forwarded to port 8082
/quality requests are forwarded to port 8081

This approach allows multiple services to be accessed through a single public endpoint.

```
server {
    server_name api.springforge.dev;

    location /code/ {
        proxy_pass http://127.0.0.1:8000/;
    }

    location /quality/ {
        proxy_pass http://127.0.0.1:8081/;
    }

    location /runtime/ {
        rewrite ^/runtime/?(.*)$ /$1 break;
        proxy_pass http://127.0.0.1:8082/;
    }

    location /website/ {
        client_max_body_size 500M;
        rewrite ^/website/(.*)$ /$1 break;
        proxy_pass http://127.0.0.1:8085/;
    }

    listen 443 ssl; # managed by Certbot
    ssl_certificate /etc/letsencrypt/live/api.springforge.dev/fullchain.pem; # managed by Certbot
    ssl_certificate_key /etc/letsencrypt/live/api.springforge.dev/privkey.pem; # managed by Certbot
    include /etc/letsencrypt/options-ssl-nginx.conf; # managed by Certbot
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem; # managed by Certbot

}
```

8. Continuous Integration and Deployment Pipeline

A CI/CD pipeline is implemented using GitHub Actions to automate the deployment process.

Pipeline Stages

- Source Code Commit
  Developers push code changes to the GitHub repository.
- Docker Image Build
  The pipeline builds Docker images using the updated source code.
- Container Registry Push
  Built images are pushed to the container registry.
- Deployment Trigger
  The pipeline connects to the virtual machine using SSH.
- Service Update
  Docker Compose pulls the latest images and restarts the containers.

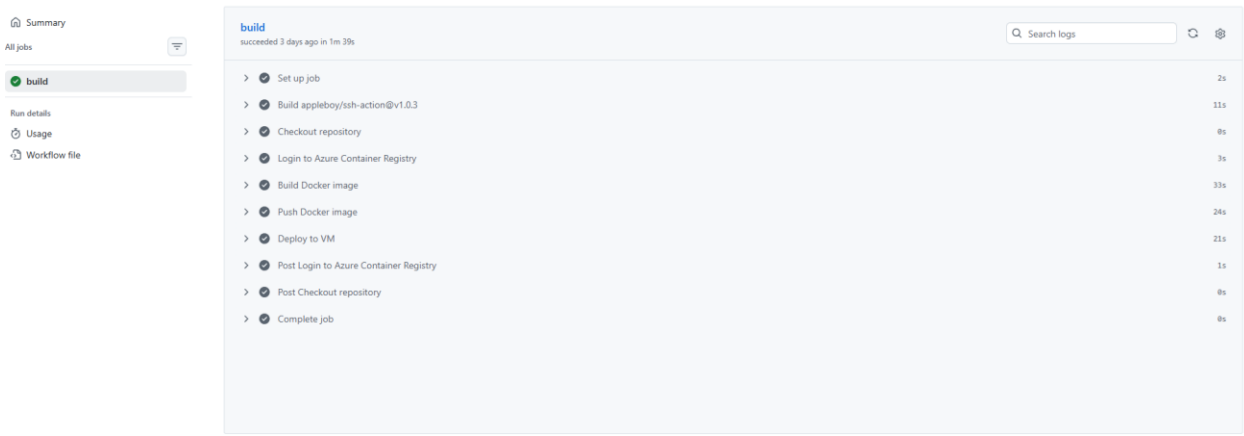
Deployment Commands Executed

- Navigate to deployment directory
- Pull latest container images
- Stop existing containers
- Start updated containers
- Remove unused images

This process ensures that the backend services are automatically updated after every successful code merge.

```yaml
name: Docker CI/CD

on:
  push:
    branches: [main]
  workflow_dispatch:

concurrency:
  group: docker-build
  cancel-in-progress: true

jobs:

  build:

    runs-on: ubuntu-latest

    steps:

    - name: Checkout repository
      uses: actions/checkout@v4

    - name: Login to Azure Container Registry
      uses: docker/login-action@v3
      with:
        registry: ${{ secrets.ACR_LOGIN_SERVER }}
        username: ${{ secrets.ACR_USERNAME }}
        password: ${{ secrets.ACR_PASSWORD }}

    - name: Build Docker image
      run: |
        docker build \
        -t ${{ secrets.ACR_LOGIN_SERVER }}/code-quality:latest \
        .

    - name: Push Docker image
      run: |
        docker push ${{ secrets.ACR_LOGIN_SERVER }}/code-quality:latest

    - name: Deploy to VM
      uses: appleboy/ssh-action@v1.0.3
```

```
      with:
        host: ${{ secrets.VM_HOST }}
        username: ${{ secrets.VM_USER }}
        key: ${{ secrets.VM_SSH_KEY }}
        script: |
          cd ~/springforge-development
          docker compose pull
          docker compose down --remove-orphans
          docker compose up -d
          docker system prune -f
```



9. Environment Configuration

Application configuration values are managed through environment variables.

Examples of configuration variables include

- Database host
- Database credentials
- Vector database URL
- Cloud service parameters
- Language model configuration

These variables are stored in environment files on the virtual machine and injected into containers during deployment.

This approach prevents sensitive credentials from being stored in the source code repository.

```
dushxn@VM-SpringForge:~/springforge-development$ ls
docker-compose.yml  quality.env  rag.env  website.env
dushxn@VM-SpringForge:~/springforge-development$
```

10. Logging and Monitoring

Docker logs are used to monitor the runtime behavior of containers.

Logs can be accessed using command line tools.

Example commands

- View container logs
  docker logs container_name
- Stream logs in real time
  docker logs -f container_name

These logs help identify runtime errors and deployment issues.

```
dushxn@VM-SpringForge:~/springforge-development$ docker ps
CONTAINER ID   IMAGE                                          COMMAND                  CREATED       STATUS       PORTS                                          NAMES
0e7dade804d1   springforgeacr.azurecr.io/website:latest       "docker-entrypoint.s…"   2 hours ago   Up 2 hours   0.0.0.0:8085->4000/tcp, [::]:8085->4000/tcp    website-backend
ca47765e5949   springforgeacr.azurecr.io/rag-error-engine:latest   "gunicorn --bind 0.0…"   2 hours ago   Up 2 hours   0.0.0.0:8082->8000/tcp, [::]:8082->8000/tcp    rag-error-engine
61886824e9c0   springforgeacr.azurecr.io/code-quality:latest  "uvicorn app.main:ap…"   2 hours ago   Up 2 hours   0.0.0.0:8081->8081/tcp, [::]:8081->8081/tcp    code-quality
19cc6a2986de   springforgeacr.azurecr.io/codegeneration-backend:latest   "uvicorn main:app --…"   2 hours ago   Up 2 hours   0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp    architecture-backend
dushxn@VM-SpringForge:~/springforge-development$ docker logs website-backend
[Nest] 1  - 03/08/2026, 3:19:19 PM     LOG [NestFactory] Starting Nest application...
[Nest] 1  - 03/08/2026, 3:19:19 PM     LOG [InstanceLoader] AppModule dependencies initialized +168ms
[Nest] 1  - 03/08/2026, 3:19:19 PM     LOG [InstanceLoader] DatabaseModule dependencies initialized +0ms
[Nest] 1  - 03/08/2026, 3:19:19 PM     LOG [InstanceLoader] StorageModule dependencies initialized +0ms
[Nest] 1  - 03/08/2026, 3:19:19 PM     LOG [InstanceLoader] HealthModule dependencies initialized +7ms
[Nest] 1  - 03/08/2026, 3:19:19 PM     LOG [InstanceLoader] PluginModule dependencies initialized +5ms
[Nest] 1  - 03/08/2026, 3:19:19 PM     LOG [InstanceLoader] AdminModule dependencies initialized +0ms
[Nest] 1  - 03/08/2026, 3:19:19 PM     LOG [InstanceLoader] FeedbackModule dependencies initialized +0ms
[Nest] 1  - 03/08/2026, 3:19:19 PM     LOG [InstanceLoader] AuthModule dependencies initialized +1ms
[Nest] 1  - 03/08/2026, 3:19:19 PM     LOG [RoutesResolver] AuthController {/api/auth}: +36ms
[Nest] 1  - 03/08/2026, 3:19:19 PM     LOG [RouterExplorer] Mapped {/api/auth/register, POST} route +19ms
[Nest] 1  - 03/08/2026, 3:19:19 PM     LOG [RouterExplorer] Mapped {/api/auth/login, POST} route +3ms
[Nest] 1  - 03/08/2026, 3:19:19 PM     LOG [RouterExplorer] Mapped {/api/auth/me, GET} route +4ms
[Nest] 1  - 03/08/2026, 3:19:19 PM     LOG [RoutesResolver] PluginController {/api/plugin}: +0ms
[Nest] 1  - 03/08/2026, 3:19:19 PM     LOG [RouterExplorer] Mapped {/api/plugin/info, GET} route +1ms
[Nest] 1  - 03/08/2026, 3:19:19 PM     LOG [RouterExplorer] Mapped {/api/plugin/download, GET} route +3ms
[Nest] 1  - 03/08/2026, 3:19:19 PM     LOG [RoutesResolver] AdminController {/api/admin}: +2ms
[Nest] 1  - 03/08/2026, 3:19:19 PM     LOG [RouterExplorer] Mapped {/api/admin/upload, POST} route +2ms
[Nest] 1  - 03/08/2026, 3:19:19 PM     LOG [RoutesResolver] FeedbackController {/api/feedback}: +0ms
[Nest] 1  - 03/08/2026, 3:19:19 PM     LOG [RouterExplorer] Mapped {/api/feedback, POST} route +1ms
[Nest] 1  - 03/08/2026, 3:19:19 PM     LOG [RouterExplorer] Mapped {/api/feedback/mine, GET} route +5ms
[Nest] 1  - 03/08/2026, 3:19:19 PM     LOG [RoutesResolver] HealthController {/api/health}: +1ms
[Nest] 1  - 03/08/2026, 3:19:19 PM     LOG [RouterExplorer] Mapped {/api/health, GET} route +2ms
[Nest] 1  - 03/08/2026, 3:19:19 PM     LOG [DatabaseService] Database schema ready
[Nest] 1  - 03/08/2026, 3:19:20 PM     LOG [NestApplication] Nest application successfully started +6ms
SpringForge backend running on http://localhost:4000
[Nest] 1  - 03/08/2026, 3:22:05 PM     LOG [AzureStorageService] Uploaded blob: 1772983324319-SpringForge-CodeGeneration-1.0-SNAPSHOT.zip
[Nest] 1  - 03/08/2026, 3:22:05 PM     LOG [AzureStorageService] Deleted blob: 1772957576911-SpringForge-CodeGeneration-1.0-SNAPSHOT.zip
dushxn@VM-SpringForge:~/springforge-development$
```

11. Security Considerations

Several security practices are implemented during deployment.

SSH key-based authentication is used for remote access to the virtual machine.

Sensitive configuration values are stored outside the source code.

Only necessary network ports are exposed.

HTTPS can be enabled using SSL certificates generated through Certbot.

12. Conclusion

The deployment architecture of SpringForge provides a scalable and automated infrastructure for hosting multiple backend microservices. Containerization ensures consistent runtime environments while CI/CD pipelines enable automated updates. The use of reverse proxy routing allows multiple services to operate under a unified endpoint while maintaining modularity and scalabilit